

1)virtual_address

```
sahasrasagi@Sahasra: ~/OS_L  ×  +  ∨  
GNU nano 8.7.1  
#include <stdio.h>  
  
int main() {  
    int x = 10;  
  
    printf("Value of x = %d\n", x);  
    printf("Virtual address of x = %p\n", (void *)&x);  
  
    return 0;  
}|  
  
sahasrasagi@Sahasra:~/OS_Lab/practical_$ gcc virtual_address.c -o virtual_address  
sahasrasagi@Sahasra:~/OS_Lab/practical_$ ./virtual_address  
Value of x = 10  
Virtual address of x = 0x7fff700f07d4  
sahasrasagi@Sahasra:~/OS_Lab/practical_$ |
```

2)memory_segments.c

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address : %p\n", (void *)main);
    printf("Data address : %p\n", (void *)&global);
    printf("BSS address : %p\n", (void *)&bss_variable);
    printf("Heap address : %p\n", (void *)heap);
    printf("Stack address : %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

```
sahasrasagi@Sahasra:~/OS_Lab/practical_$ ./memory_segments
Code address : 0x6540f68c21a9
Data address : 0x6540f68c5010
BSS address : 0x6540f68c5018
Heap address : 0x654115405010
Stack address : 0x7ffc7f1615fc
sahasrasagi@Sahasra:~/OS_Lab/practical_$ |
```

3)address_translation

```
sahasrasagi@Sahasra: ~/OS_L × + v
GNU nano 8.7.1 a
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number      = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}|

sahasrasagi@Sahasra:~/OS_Lab/practical_$ nano address_translation.c
sahasrasagi@Sahasra:~/OS_Lab/practical_$ gcc address_translation.c -o address_t
sahasrasagi@Sahasra:~/OS_Lab/practical_$ ./address_translation
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number      = 1
Physical Address = 150
sahasrasagi@Sahasra:~/OS_Lab/practical_$ |
```

4)null_pointer.c

```
sahasrasagi@Sahasra: ~/OS_L × + ∨
GNU nano 8.7.1
#include <stdio.h>

int main() {

    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}|
```

```
sahasrasagi@Sahasra:~/OS_Lab/practical_$ nano null_poi
sahasrasagi@Sahasra:~/OS_Lab/practical_$ gcc null_poin
.sahasrasagi@Sahasra:~/OS_Lab/practical_$ ./null_point
Before accessing memory
Segmentation fault ./null_pointer
sahasrasagi@Sahasra:~/OS_Lab/practical_$ |
```

5)dynamic_malloc.c

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}|

sahasrasagi@Sahasra:~/OS_Lab/practical_$ nano dynamic_malloc.c
sahasrasagi@Sahasra:~/OS_Lab/practical_$ gcc dynamic_malloc.c
sahasrasagi@Sahasra:~/OS_Lab/practical_$ ./dynamic_malloc
10 20 30 40 50
sahasrasagi@Sahasra:~/OS_Lab/practical_$ |
```